

Lec 08: Application layer: web & HTTP

4/9/23

* Application layer architectures

- Client/server

client

server

- * always ON
- * high capacity
 - compute, BW, memory, ...

- Peer-to-Peer (P2P)

• e.g. Torrent

• -



* Socket interface: TCP vs UDP semantics

* Application types: elastic vs. real time

* HTTP: Hypertext Transfer Protocol

- connⁿ oriented protocol
- transport layer: TCP
- stateless protocol

- request-response protocol

Stateful
server

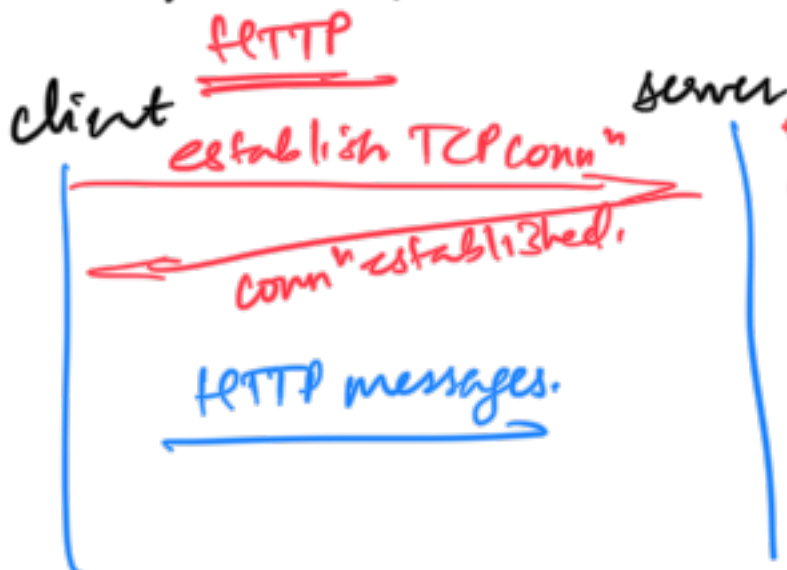
- server stores the request state

- server is bulky

Stateless
server

- server does not store any client state

- n/w BW is wasted
-- req. size is large



socket, bind
listen -- 80

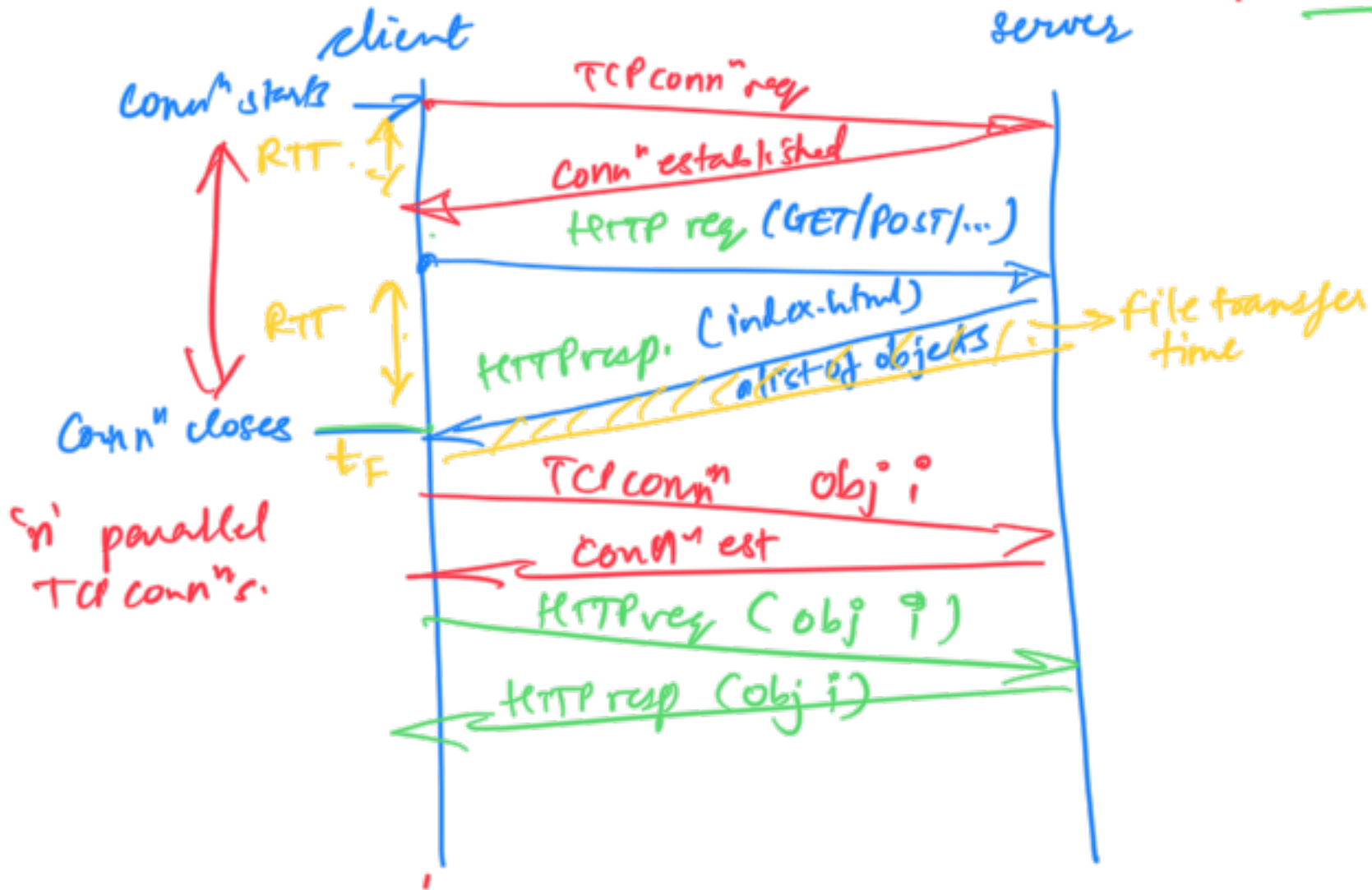
* HTTP connections $\Rightarrow$ 2 TYPES

• Non-persistent HTTP connection

HTTP/1.0

IP addr

abc.com/xyz/index.html
server

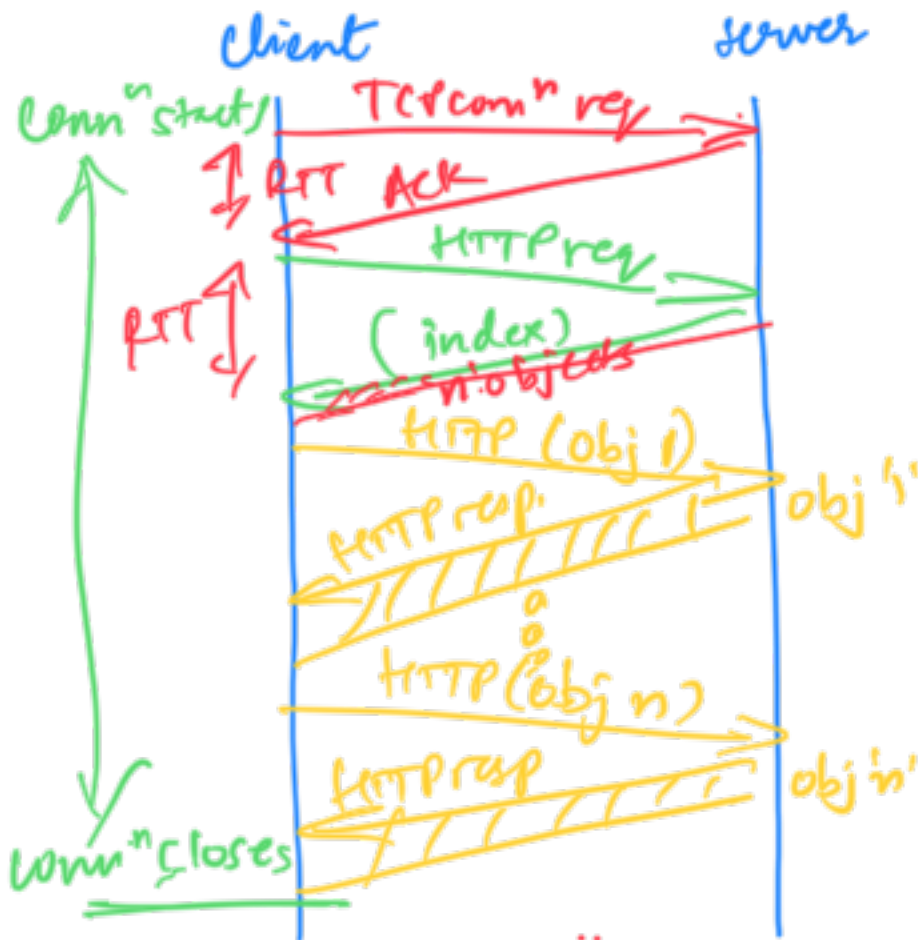


'n' objects

$$\text{Page load time} = 2RTT + t_F(\text{index}) + 2RTT + \text{object download time of max sized object}$$

- Persistent HTTP connection default for HTTP/1.1

- without pipelining
sequential req

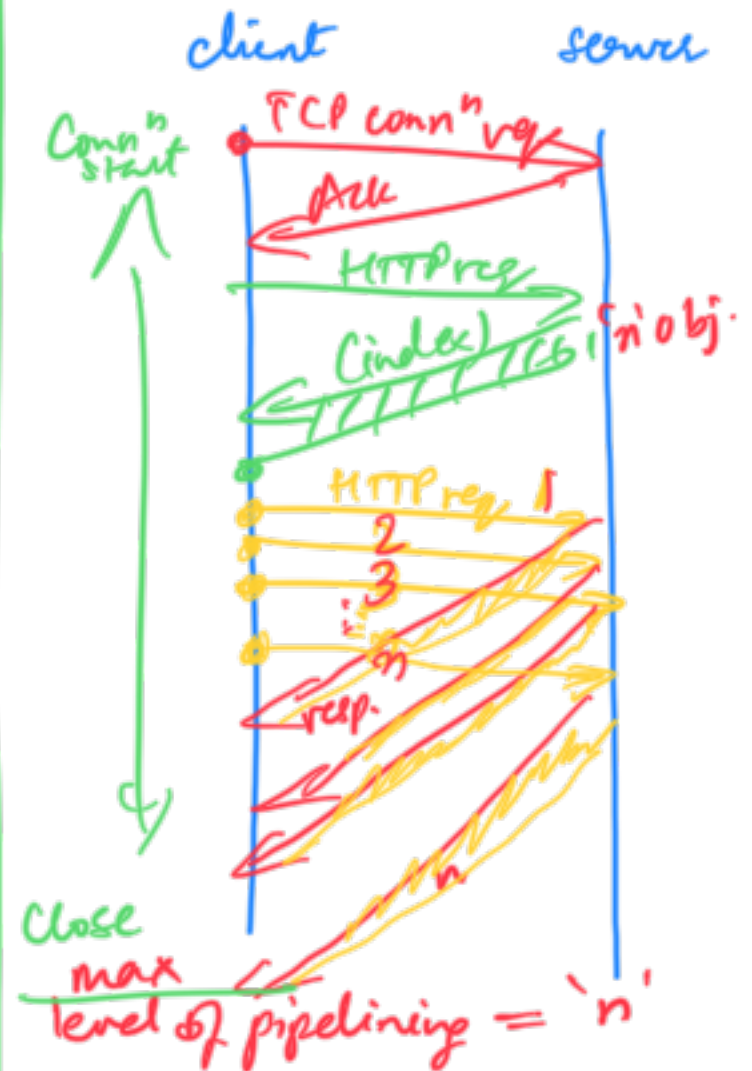


Page load time =

$$2RTT + t_f(\text{index}) + n * (RTT + t_{o_i}(\text{object download time}))$$

→ head blocking ... head-of-line

- with pipelining



Page load time =

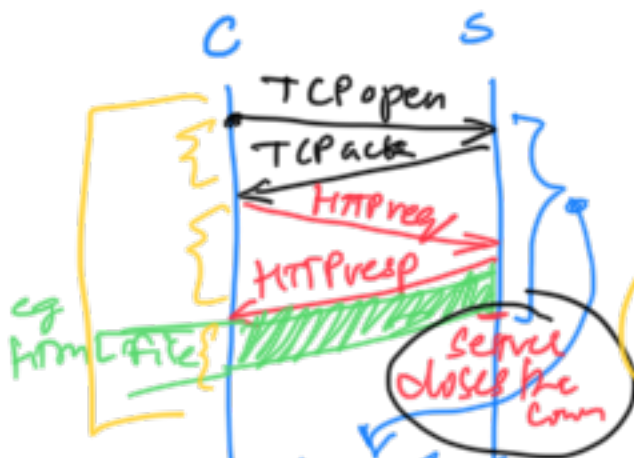
$$2RTT + t_f(\text{index}) + \left\lceil \frac{n}{\text{max pipelining level allowed}} \right\rceil * (RTT + \text{max object download time})$$

→ use ceil value

→ # parallel requests depends on the parallelism level supported by the browser

SUMMARY

A HTTP/1.0 Non-persistent HTTP



Repeat above steps for each object to be requested

NOTE: The objects can be requested in PARALLEL

- Too many conn's; Loaded server

Time = $2 * RTT + \text{file transfer time}$ (for index.html)

+ $2 * RTT + \text{file transfer time of largest file}$

For 'n' parallel objects

B Persistent HTTP without pipelining



- Single TCP conn for all objects
- requests sent in the order as an object is encountered
↳ cannot send next req until the previous request's response is received
↳ HOL blocking

Time = $2 * RTT + t_{file}$
+ $n * (RTT + t_{file})$

C HTTP/1.1 Persistent HTTP with pipelining



- Single TCP conn for all objects
- Req are sent in parallel w/o waiting for prev response

- HOL blocking exists
- solved by HTTP/2.0
- HTTP/3.0 addresses security, congestion control, ... over UDP

extra info

Time = $2 * RTT + t_{file}$

+ $\left[\frac{n}{\text{pipelining levels}} \right] * (RTT + \text{largest of } (t_{file1}, t_{file2}, \dots, t_{file n}))$
largest file transfer time

* NOTE:
parallel requests depends on level of parallelism supported by the client browser.

TCP sockets $\Rightarrow$ Synchronous
client server

